

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

---

### COURSE OUTCOME COVERED

*CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)*

*Assessment Tool Weightage: 20%*

---

QUESTIONS: 3

Total Marks:50

Annexure A

### ERROR ANALYSIS TASK

---

#### *Code of Conduct:*

I V.santhosh kumar (192411015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

*I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: v.santhosh kumar

## Annexure A

### ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>
```

```
int sumArray(int arr[], int n) {  
    int i, sum = 0;  
    for(i = 0; i <= n; i++) {  
        sum = sum + arr[i];  
    }  
    return sum;  
}
```

```
int main() {  
    int arr[5] = {1,2,3,4,5}  
    int total;  
  
    total = sumArray(arr);  
  
    if(total == 15) {  
        printf("Correct Sum\n")  
    }  
  
    int a = 10, b = 0;
```



Syntax: missing semicolons

Semantic: wrong function call

Logical:  $i \leq n$

Runtime: divide by zero, invalid pointer

(c) Compile time: syntax & semantic. Execution: logical/runtime.

(d) Corrections: `for(i=0;i<n;i++)`; arr declaration with `;` `total=sumArray(arr,5)`; `printf(...)`;  
use `b=2`; `int *p=&total`;

(e) Impacts: pointer crash, divide-by-zero undefined behavior, array overflow.

(f) Improvements: better diagnostics, static analysis.

Q2. The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>
```

```
int main() {  
    int 1num = 100;  
    float rate@ = 25.5;  
    char ch = 'abc';  
    int total = 50$20;  
    int num#1 = 30;  
    int %count = 40;  
  
    printf("Welcome to Compiler Design);  
  
    int x = 09;  
    int y = 0xZ12;  
    int z = 12.3.4;  
  
    char str1[] = "Hello;  
    char str2[] = "World";  
  
    int _valid = 10;  
    int invalid-id = 20;  
  
    float value = 5..6;  
    int data = 10@5;  
  
    char c = 'xy';  
    int 3value = 60;  
  
    printf("Value: %d\n", total);
```

```
    return 0;  
}
```

#### Tasks

##### (a) Error Identification

- i) Identify all lexical errors in the program.
- ii) List the line number and the invalid token.

##### (b) Error Classification

- i) For each error, explain why it is considered a lexical error.
- ii) Specify which rule of token formation is violated:
- iii) Identifier rules
- iv) Constant formation
- v) String/character literal rules
- vi) Operator/symbol rules

##### (c) Token Correction



**(b) Classify by Compiler Phase**

Classify each error into:

- I. Lexical (invalid symbols like @num)
- II. Syntax (missing semicolon, brackets)
- III. Semantic (function argument mismatch)
- IV. Intermediate Code Issues (wrong expression handling/precedence)
- V. Optimization Issues (redundant + 0)
- VI. Runtime Errors (division by zero, invalid pointer access)

**(c) Compilation vs Execution (Evaluation)**

- I. Which errors are detected at **compile time**?
- II. Which errors appear only during **execution**?
- III. Explain why some errors prevent further compilation.













